CUDA_Experiment_8\array_max.cu

```
#include <stdio.h>
#include <cuda_runtime.h>

#define CUDA_CHECK(call) do { \
    cudaError_t err__ = (call); \
    if (err__ != cudaSuccess) { \
        fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__, cudaGetErrorString(err__)); \
        return 1; \
    } \
} while (0)

__global__ void findMax(const int *input, int *result, int N) {
    // Each thread handles one element; atomicMax safely combines candidates.
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) atomicMax(result, input[i]);
}

int main() {
    const int N = 8;
    int h_A[N] = {12, 7, 25, 3, 19, 42, 8, 31};
    int h_max = h_A[0];
    int *d_A = NULL, *d_max = NULL;

    CUDA_CHECK(cudaMalloc((void**)&d_A, N * sizeof(int)));
    CUDA_CHECK(cudaMalloc((void**)&d_max, sizeof(int)));
    CUDA_CHECK(cudaMemcpy(d_A, h_A, N * sizeof(int), cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_max, &h_max, sizeof(int), cudaMemcpyHostToDevice));

    int threads = 256;
    int blocks = (N + threads - 1) / threads;
    findMax<<<blocks, threads>>>(d_A, d_max, N);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaMemcpy(&h_max, d_max, sizeof(int), cudaMemcpyDeviceToHost));

    printf("Array: ");
    for (int i = 0; i < N; ++i) printf("%d ", h_A[i]);
    printf("\nMaximum value = %d\n", h_max);

    cudaFree(d_A); cudaFree(d_max);
    return 0;
}

```